

SECURE CODING REVIEW

Submitted By

Name: Uppala Pravallika

Department: Computer Science and Engineering

College: Rajiv Gandhi University Of Knowledge And Technologies

Date: 18-06-2026

1. Introduction

Secure coding is the practice of writing software in a way that protects applications from security vulnerabilities and cyber threats. The objective of this assignment is to review a Java-based application, identify security weaknesses, and recommend appropriate remediation measures to improve software security.

2. Selected Programming Language and Application

Programming Language

Java

Application

Login System

Application Description

The Login System allows users to enter a username and password to access the application. The review focuses on identifying vulnerabilities related to authentication, password management, and input handling.

Sample Code

```
import java.util.Scanner;

public class LoginSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Username: ");
        String username = sc.nextLine();

        System.out.print("Password: ");
        String password = sc.nextLine();

        if(username.equals("admin") && password.equals("admin123")) {
            System.out.println("Login Successful");
        } else {
            System.out.println("Login Failed");
        }
    }
}
```

3. Code Review Process

The source code was reviewed using manual inspection methods. The review focused on:

- Authentication mechanisms
- Password storage and handling
- Input validation
- Security best practices
- Protection against common attacks

The identified vulnerabilities were analyzed based on their impact and severity.

4. Security Findings

Finding 1: Hardcoded Credentials

Description

The username and password are directly stored within the source code.

Risk Level

High

Impact

An attacker who gains access to the source code can easily view the credentials and access the application.

Remediation

Store credentials securely in a database and avoid hardcoding sensitive information.

Finding 2: Plain Text Password Handling

Description

Passwords are stored and compared in plain text.

Risk Level

High

Impact

If credentials are exposed, attackers can immediately use them.

Remediation

Use secure hashing algorithms such as BCrypt to store passwords.

Finding 3: Missing Input Validation

Description

User inputs are accepted without validation.

Risk Level

Medium

Impact

Invalid or malicious inputs may affect application behavior and security.

Remediation

Validate and sanitize all user inputs before processing.

Finding 4: Unlimited Login Attempts

Description

The application allows unlimited failed login attempts.

Risk Level

Medium

Impact

Attackers may perform brute-force attacks to guess passwords.

Remediation

Implement account lockout or rate-limiting mechanisms after multiple failed login attempts.

5. Vulnerability Summary

Vulnerability	Risk Level	Recommendation
Hardcoded Credentials	High	Store credentials securely
Plain Text Passwords	High	Use BCrypt hashing
Missing Input Validation	Medium	Validate user inputs
Unlimited Login Attempts	Medium	Implement account lockout

6. Recommendations and Best Practices

The following secure coding practices are recommended:

1. Avoid hardcoding usernames, passwords, and API keys.
 2. Store passwords using secure hashing algorithms.
 3. Validate and sanitize all user inputs.
 4. Use secure authentication mechanisms.
 5. Apply the principle of least privilege.
 6. Keep software libraries and dependencies updated.
 7. Conduct regular security testing and code reviews.
 8. Implement logging and monitoring mechanisms.
-

7. Conclusion

The secure coding review of the Java Login System identified several security vulnerabilities, including hardcoded credentials, insecure password handling, missing input validation, and the lack of protection against brute-force attacks. These issues may lead to unauthorized access and compromise the security of the application.

By implementing the recommended remediation measures and following secure coding best practices, the security and reliability of the application can be significantly improved.

8. References

1. Oracle Java Documentation

2. OWASP Secure Coding Practices
3. OWASP Top 10 Security Risks
4. Secure Coding Guidelines for Java